EXPERIMENT-8

Write a program for Travelling Salesman Problem using python code

Aim : To solve the **Travelling Salesman Problem (TSP)** using Python.

Procedure:

Define the distance matrix between cities.
Generate all possible routes.
Calculate the total distance for each route.
Select the route with the minimum distance.
Display the shortest route and its cost.

PYTHON CODE:

```
from itertools import permutations

cities = ['A', 'B', 'C', 'D']

distance = [

    [0, 10, 15, 20],

    [10, 0, 35, 25],

    [15, 35, 0, 30],

    [20, 25, 30, 0]

]

start = 0

min_cost = float('inf')

best_route = []

for route in permutations(range(1, len(cities))):

    cost = 0

    current = start

    for city in route:

        cost += distance[current][city]

        current = city

    cost += distance[current][start]
```

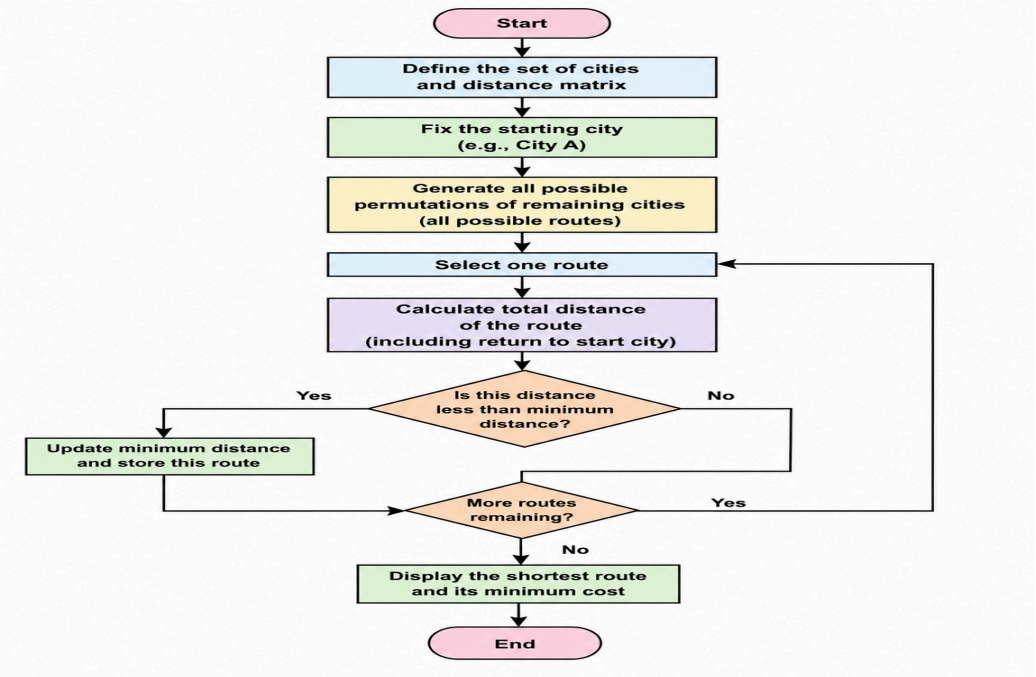
```
    if cost < min_cost:
        min_cost = cost
        best_route = route
print("Shortest Route:")
print(cities[start], end=" -> ")
for city in best_route:
    print(cities[city], end=" -> ")
print(cities[start])
print("Minimum Cost =", min_cost)
```

OUTPUT:

```
from itertools import permutations

graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

cities = [1, 2, 3]
min_cost = float('inf')
best_path = ()

for path in permutations(cities):
    cost = graph[0][path[0]]
    for i in range(len(path)-1):
        cost += graph[path[i]][path[i+1]]
    cost += graph[path[-1]][0]

    if cost < min_cost:
        min_cost = cost
        best_path = path

print("Shortest Path: 0 ->", *best_path, "-> 0")
print("Minimum Cost:", min_cost)
```

```
Shortest Path: 0 -> 1 3 2 -> 0
Minimum Cost: 80

=== Code Execution Successful ===
```

FLOWCHART:



Result: Hence, Travelling Salesman problem has solved